

INTEGRATED PROJECT · KM-04 + KM-05

Thabo's Corner Store

Sales Insights System — Data, Databases & Programming in Practice

Project Brief & Submission Instructions

Occupational Certificate: Artificial Intelligence Software Developer · NQF Level 4

Covers: KM-04 Data & Databases + KM-05 Programming & Python

1. Project Overview

This is an integrated, hands-on project that brings together everything you have learned across two knowledge modules: KM-04 (Data & Databases) and KM-05 (Programming & Python). Instead of practising these skills separately, you will use them together — exactly the way a working AI Software Developer would — to build a small but complete data application.

You will take messy, real-world-style raw data, clean it, store it properly in a relational database, write Python code to analyse it, turn your analysis into charts, add a basic security gate, and document your thinking throughout. Finally, you will submit your work as evidence in two places: a zip file on UpSkill, and a version-controlled repository on GitHub.

Why this matters:

Employers and assessors don't just want to see that you can follow instructions — they want to see a working project with a visible history of how it was built. That's exactly what a GitHub repository with regular commits demonstrates, and exactly why it forms part of your evidence.

2. Learning Outcomes Covered

This project gives you evidence against topic elements from both knowledge modules:

Module	Topic Element	Where it shows up in this project
KM-04	KT01 Introduction to Data	Identifying data types and errors of omission in the raw CSV files
KM-04	KT02 Data in Spreadsheets	Working with the raw CSV data before it enters the database
KM-04	KT03 Data Analytics	Interpreting trends and patterns from your SQL query results
KM-04	KT04 Introduction to Databases	Designing the products/sales relational schema in SQLite
KM-04	KT06 Structured Query Language (SQL)	Writing JOIN, GROUP BY and aggregation queries
KM-04	KT07 Visualising Data	Producing bar and line charts with matplotlib
KM-04	KT08 Data Security	Building a password-hashing login gate; CIA-triad reflection
KM-05	KT01/KT02 Languages & Algorithms	Planning your solution with computational thinking before coding
KM-05	KT03 Programming Basics	Functions, loops, conditionals and lists in analysis.py
KM-05	KT04 Solution Development	Decomposing the project into the five task modules
KM-05	KT05 Introduction to Python	The entire project is written in Python

3. The Scenario

Thabo runs a small corner store (a 'spaza shop') and has been recording every sale in a spreadsheet for the past few months. He knows there is useful information hiding in that data — which products sell best, which days are busiest, where he's losing track of stock — but the data is messy, it lives in disconnected files, and he has no way to protect it from prying eyes.

You have been hired as a junior AI Software Developer to build him a small system that cleans up his data, stores it properly, analyses it, shows him the results as charts, and keeps his sales reports behind a simple password.

4. Resources Provided

You have been given a project package containing the following:

- data/products.csv — raw product data (14 products) with realistic data-quality issues you must clean.
- data/sales.csv — raw sales transaction data (71 records) with realistic data-quality issues you must clean.
- starter_code/ — a Python project scaffold with five modules and TODO comments (see Section 6).
- This Project Brief document.
- The GitHub & VS Code Setup Guide document.

4.1 products.csv — column reference

Column	Description
product_id	Unique product code, e.g. P001. Used to link to sales.csv.
product_name	Name of the product. A few rows are missing this — decide how to handle it.
category	Product category, e.g. Beverages, Snacks. Capitalisation is inconsistent in the raw file.
unit_price	Price in Rand. A few rows are missing this value.
in_stock	True / False — whether the product is currently in stock.

4.2 sales.csv — column reference

Column	Description
sale_id	Unique transaction code, e.g. S0001.
product_id	Links to products.csv. A few rows reference a product_id that does not exist — a data-entry error you must catch.
quantity	Number of units sold. A few rows are missing this value.
sale_date	Date of the sale. Deliberately written in at least four different formats, and sometimes missing entirely.
payment_method	Cash / Card / Mobile Money. A few rows are missing this value.
customer_type	Walk-in / Regular. Optional context field.

Hint — the date problem is deliberate:

Some dates in sales.csv could genuinely be read two different ways (for example, 05-01-2026 could mean 5 January or January the 5th, depending on the format). This is a classic real-world data problem. Document, in your README, which format you standardised on and why — there is no single 'correct' answer, but there is a clearly wrong approach: guessing silently and not telling anyone.

5. Tools & Software You Need

- Python 3.10 or later
- Visual Studio Code (VS Code)
- Git (version control)
- A free GitHub account
- Python packages: pandas, matplotlib (installed via requirements.txt — see starter_code/README.md)
- Optional but recommended: the 'DB Browser for SQLite' free app, to visually inspect your store.db file

Full setup instructions for Git, GitHub and VS Code are in the separate GitHub & VS Code Setup Guide document included in your project package.

6. Project Tasks

Complete the following eight tasks, in order. Each task builds on the one before it.

Task 1 — Plan Before You Code (Computational Thinking)

Before writing any code, apply the four pillars of computational thinking from KM-05: decompose the problem, spot patterns, decide what to abstract away, and sketch your algorithm.

Write a short plan (half a page is enough) covering: what the biggest sub-problems are, what you expect to find messy in the data, and the order you intend to tackle the five starter_code modules in.

Evidence for this task:

A PLAN.md or PLAN.docx file with your written plan, added to your repo before you start coding.

Task 2 — Clean the Data

Complete the functions in data_cleaning.py. You must handle every issue listed in Section 4: missing values, inconsistent categories, inconsistent date formats, invalid product references, and duplicate rows.

For every decision you make (e.g. 'rows with a missing sale_date were dropped'), write one sentence explaining why in your README findings section.

Evidence for this task:

Completed data_cleaning.py; a findings paragraph in README.md explaining your cleaning decisions.

Task 3 — Build the Relational Database

Complete database.py: design and create the products and sales tables (with appropriate primary and foreign keys), insert your cleaned data, and write at least five working SQL queries — including at least one JOIN and one aggregation (SUM, COUNT or AVG).

Two extra queries (query_custom_one and query_custom_two) are yours to design — pick business questions Thabo would actually care about.

Evidence for this task:

Completed database.py; the generated store.db file; your five+ SQL queries and their results pasted into README.md.

Task 4 — Programming Logic

Complete analysis.py using plain Python — functions, loops, conditionals and lists/dictionaries, not just pandas one-liners. This is where you demonstrate KM-05 programming basics directly.

Evidence for this task:

Completed analysis.py, producing correct output when run from main.py.

Task 5 — Visualise Your Findings

Complete `visualize.py` to produce at least two charts: a bar chart of revenue per product (or category) and a line chart of the monthly sales trend.

Evidence for this task:

Two `.png` chart files, and a short paragraph in `README.md` describing what each chart shows.

Task 6 — Add a Data Security Gate

Complete `security.py` so that reports cannot be viewed without a correct password, using SHA-256 hashing (never store or compare plain-text passwords).

Answer the three CIA-triad reflection questions at the bottom of `security.py` in your `README` — in your own words, referencing your own design choices in this project.

Evidence for this task:

Completed `security.py`; a CIA-triad reflection section in `README.md` (150–250 words).

Task 7 — Document Everything

Update `starter_code/README.md` with: how to install and run your project, your data-cleaning decisions, your SQL queries and results, your chart descriptions, your CIA-triad reflection, and a short 'Key Business Insights' summary (roughly 200–300 words) written for Thabo, who is not a programmer.

Evidence for this task:

A complete, well-organised `README.md`.

Task 8 — Version Control & GitHub Submission

Push your project to a GitHub repository, committing your progress regularly as you complete each task above — not as one single commit at the end. Aim for at least 8 commits with clear, specific messages (e.g. 'Complete data cleaning for `products.csv`', not 'update').

Follow the GitHub & VS Code Setup Guide if you have not connected VS Code to GitHub before.

Evidence for this task:

A public or unlisted GitHub repository link, with visible commit history, submitted alongside your zip file (see Section 8).

7. Using GitHub as Evidence

A GitHub repository with a visible history of commits is one of the strongest pieces of evidence you can submit — it proves the work is genuinely yours, built up over time, not assembled or copied the night before the deadline.

- Create your repository BEFORE you start Task 2, not after you've finished.
- Commit after each task, or even after each meaningful sub-step within a task.
- Write commit messages that describe what changed, e.g. 'Add SQL queries for revenue and best seller'.
- Do not commit your virtual environment folder (venv/) — the setup guide shows you how to exclude it with .gitignore.

Full step-by-step instructions — from creating your GitHub account to pushing your first commit from VS Code — are in the separate GitHub & VS Code Setup Guide.

8. What to Submit on UpSkill

Prepare a single zip file containing:

- Your completed starter_code/ folder (all five modules plus main.py), with the TODOs completed.
- Your PLAN.md from Task 1.
- Your completed README.md from Task 7.
- Your two .png chart files from Task 5.
- Your store.db database file from Task 3.
- A short SUBMISSION.txt file containing only your full name, student number, and your GitHub repository URL.

Naming convention: Name your zip file Surname_Firstname_KM04-05_Project.zip (e.g. Dlamini_Thabo_KM04-05_Project.zip).

Submission steps

1. Confirm your project runs end-to-end with python main.py and produces no errors.
2. Confirm your GitHub repository is up to date — your last commit should match what's in your zip file.
3. Select all the required files and folders listed above and compress them into one .zip file.
4. Log in to UpSkill and navigate to the KM-04 / KM-05 Integrated Project assignment.
5. Upload your zip file using the file submission tool.

6. Paste your GitHub repository link into the submission comment / text box provided.
7. Click Submit, and check that you receive an on-screen or email confirmation from UpSkill.

Deadline reminder:

Check UpSkill or your facilitator for the exact due date and time for this assignment. Late submissions should still include your GitHub link — a repository with commits made before the deadline is strong evidence even if your zip upload is delayed.

9. Assessment Rubric

Criteria	Weight	What we're looking for
1. Data Cleaning (KM-04)	15%	Missing values, duplicates and inconsistent formats identified and handled sensibly, with decisions justified.
2. Database & SQL (KM-04)	20%	Correct relational schema with keys; five or more working SQL queries including a JOIN and an aggregation.
3. Programming Logic (KM-05)	20%	Correct, working use of functions, loops, conditionals and lists; code runs without errors.
4. Data Visualisation (KM-04)	10%	At least two clear, correctly labelled charts that genuinely support the findings.
5. Data Security (KM-04)	10%	Working password-hashing login gate; thoughtful, specific CIA-triad reflection.
6. Documentation & Insights	10%	Clear README; plain-language business insights; computational-thinking plan included.
7. Version Control / GitHub	15%	Multiple meaningful commits over time; working repository link; sensible commit messages.
Total	100%	

10. Academic Integrity & Getting Help

You may discuss approaches with classmates and search for help online — that's normal software development practice. However, the code, analysis, README and commit history you submit must reflect your own understanding and your own work.

If you get stuck, that's expected — this is a learning project. Ask your facilitator, and use your commit history to show what you tried, even if it didn't work. Partial, honest progress with clear commits is worth more than a perfect result you can't explain.